

A* → CoT Rescue Distillation

A minimal 2-day LoRA/QLoRA experiment plan

Prepared for your current CoT vs A* reasoning setup

Scope: one dataset, one base model, rescue-rationale rewriting, LoRA fine-tuning, and a clean evaluation in 48 hours.

Goal	Show that some CoT failures can be repaired by distilling successful A* traces into short linear rationales.
Recommended minimal scope	Use one dataset first (StrategyQA recommended), one already-working base model, and only the A*-rescued subset plus a preservation subset.
Training method	LoRA by default; QLoRA if VRAM is tight. Use TRL SFTTrainer with PEFT adapters.
Main success metric	Rescue rate on the old “CoT wrong / A* right” dev examples, while preserving performance on old “CoT right / A* wrong” cases.

Why this exact experiment

Your paper already shows the decisive signal: CoT and A* are complementary, the oracle gap is large, and the strongest problem categories are non-linear evidence aggregation, hypothesis elimination, and multi-hop chaining. The smallest publishable next step is to test whether successful A* traces can be rewritten into compact linear rationales and then distilled back into a CoT-style decoder.

- Do not try to solve everything in 2 days. Do not add mechanistic interpretability yet.
- Do not start with all datasets or all models. One dataset + one base model is enough for the first result.
- Do not distill raw A* traces directly. Rewrite them first into short linear rationales.

Anchoring assumptions from your current paper

- Oracle gains are already large: +9.79 on StrategyQA, +13.51 on LogicQA, and +14.10 on HotpotQA over the stronger single method.
- The A*-only bucket is large enough to matter: 10.8% on StrategyQA, 13.5% on LogicQA, and 17.4% on HotpotQA.
- A* helps most on non-linear evidence aggregation, hypothesis elimination, and multi-hop chaining; CoT remains stronger on direct factual reasoning and globally coherent argumentation.

1. Exact scope for the 48-hour run

Recommended scope

1. Dataset: StrategyQA first. It is the easiest minimal setting because answers are boolean, rationales can be short, and evaluation is cheap and robust.
2. Model: use the same model family that already produced your CoT/A* results if it is already integrated into your codebase. If you have A100-class hardware, use the model you already evaluated. If VRAM is tight, switch to QLoRA on the same or a slightly smaller instruction-tuned base model.
3. Train only one adapter. Do not train multiple variants on day 1.
4. Create only one training set version first: “preserve + rescue.”
5. Evaluate on one held-out dev split with four buckets: both-correct, CoT-only, A*-only, both-wrong.

Why StrategyQA first

- Short outputs reduce noise in rationale rewriting and training.
- Boolean evaluation lets you move fast and detect regressions immediately.
- Your paper already shows a meaningful A*-only bucket and a strong oracle gap there, so rescue distillation is testable.

Fallback choice

If your existing pipeline for StrategyQA is messy but HotpotQA is already clean, use HotpotQA instead. The only thing that matters for this 2-day run is implementation speed, not theoretical purity.

2. Directory layout to create before doing anything else

```
project_root/  
  data/  
    raw/  
    cot_astar_runs/  
    rescue_pairs/  
    lora_train/  
    eval/  
  prompts/  
    rewrite_prompt.txt  
    verify_prompt.txt  
  scripts/  
    build_rescue_set.py  
    rewrite_rationales.py  
    filter_rationales.py  
    make_lora_jsonl.py  
    train_lora.py  
    evaluate_rescue.py  
  outputs/  
    adapters/  
    generations/  
    metrics/  
  notes/  
    day1_log.md  
    day2_log.md
```

Non-negotiable logging files

- day1_log.md: every command you run, every count you obtain, every failure mode you notice.
- day2_log.md: exact training config, adapter path, evaluation command, and final numbers.
- rescue_manifest.csv: one row per rescued example with question id, gold label, CoT answer, A* answer, rewrite status, verification status, and train/dev assignment.

3. Data construction: preserve bucket + rescue bucket

You need two buckets, not one.

6. Preserve bucket = examples where original CoT was correct. These stop the adapter from ruining the part of CoT that already works.
7. Rescue bucket = examples where original CoT was wrong but A* was correct. These are the key supervision examples.
8. Ignore “both wrong” for the first 2-day version. Ignore “A* wrong, CoT wrong but maybe solvable” for now.

Recommended split for the minimal run

Bucket	Definition	Use in minimal run
Preserve	CoT correct	Sample a moderate subset so the adapter does not drift
Rescue	CoT wrong and A* correct	Use all available high-quality examples
Do not use yet	Both wrong	Leave out for the first run
Do not use yet	A* wrong and CoT wrong but rewriter invents something	Leave out completely

Concrete selection rule

```
rescue_set = {x : cot_pred != gold and astar_pred == gold}
preserve_set = {x : cot_pred == gold}

# good first ratio
train_examples = preserve_sample(about_same_size_as_rescue_or_2x_rescue) + rescue_set
```

Why not train only on rescue?

Because the adapter will over-specialize to the failure regime and can degrade ordinary CoT behavior. For the first run, preserving baseline competence matters almost as much as rescue.

4. Generate compact rescue rationales from successful A* traces

This is the central step. The target is not the raw A* trace. The target is a short, direct, coherent linear rationale that preserves the decisive logic found by A* but removes search artifacts.

Input fields for each rescue example

- question
- gold answer
- raw CoT trace and answer (optional but useful for debugging)
- successful A* trace and answer

Rewriting prompt template

You are given a question, the correct answer, and a successful search-style reasoning trace.
Rewrite the trace into a compact linear chain-of-thought that a single-pass reasoner could plausibly produce.

Rules:

1. Keep only logically necessary steps.
2. Remove explicit branching, search artifacts, retries, and repeated steps.
3. Do not add unsupported facts.
4. Use at most 4 numbered steps.
5. End with: Therefore, the answer is: <answer>.

Question: {question}

Correct answer: {gold}

Successful A* trace: {astar_trace}

Output only the rewritten rationale.

What makes a rewrite acceptable

- It is shorter than the source A* trace.
- It preserves the decisive bridge fact, elimination logic, or multi-hop link that made A* succeed.
- It reads like one smooth trajectory rather than a tree search log.
- Its final answer matches the gold answer exactly.

Fast verification pass

Given the question and the rewritten rationale, answer two things:

(1) Does the rationale support the stated answer? [YES/NO]

(2) Is the rationale introducing unsupported new claims not present in the successful search trace? [YES/NO]

Keep the example only if (1)=YES and (2)=NO.

Hard filters for the 2-day run

- Maximum rationale length: 120–160 tokens.
- Maximum steps: 4.
- Reject if the rewriter says “another possibility”, “branch”, “option 1”, or similar search language unless it is a multiple-choice elimination example.
- Reject if the answer extracted from the rewritten rationale does not match gold.
- Reject if the rationale is nearly identical to the original wrong CoT trace.

Minimal JSON schema after filtering

```
{
  "id": "strategyqa_train_00123",
  "question": "...",
  "gold_answer": "yes",
  "raw_cot_answer": "no",
  "raw_astar_answer": "yes",
  "rescued_rationale": "1. ...\n2. ...\nTherefore, the answer is: yes.",
  "bucket": "rescue"
}
```

5. Convert the data into LoRA training format

Recommended format: conversational prompt-completion

Use a chat-style structure so you can train only on the assistant output. TRL supports conversational data and can compute loss only on assistant messages with `assistant_only_loss=True`, provided the chat template exposes assistant token masks. If your template does not support that cleanly, then use a prompt-completion format and compute completion-only loss instead.

Simple conversational record

```
{
  "messages": [
    {"role": "system", "content": "You are a careful reasoner. Produce a short chain of thought and then the final answer."},
    {"role": "user", "content": "Question: ..."},
    {"role": "assistant", "content": "1. ...\n2. ...\nTherefore, the answer is: yes."}
  ]
}
```

Preserve examples should use the same format

For preserve examples, the assistant content should be the original good CoT if it is already compact and decent. If the baseline CoT is too long or messy, rewrite it lightly into the same short format so that train targets are stylistically consistent.

6. LoRA/QLoRA training recipe for the 2-day experiment

Choice of method

- Use plain LoRA if the model already fits comfortably in BF16/FP16 on your GPU.
- Use QLoRA if VRAM is tight or if you want the safest minimal run on a single GPU. PEFT documents the standard pattern: load the base model in 4-bit via `BitsAndBytesConfig`, call `prepare_model_for_kbit_training()`, and then attach a `LoraConfig` adapter on top. NF4 and double quantization are the standard 4-bit choices in the official guide.

Recommended adapter config for the first run

Parameter	Suggested value	Why
r	16	Strong enough for a first pass without being large
lora_alpha	16 or 32	Reasonable scaling; keep simple
lora_dropout	0.05	Small regularization
target_modules	model-specific attention projections, or "all-linear" if you want the easiest broad setup	Broad enough coverage for reasoning behavior
bias	none	Standard simple choice
task_type	CAUSAL_LM	Text generation objective

Recommended training config for the first run

Setting	First try
---------	-----------

learning_rate	1e-4
epochs	1 to 2
per_device_train_batch_size	1 to 4 depending on sequence length and GPU
gradient_accumulation_steps	adjust to reach an effective batch around 16–32 examples
max_seq_length	keep tight: 512 or 768 for StrategyQA-type data
packing	False for the first clean run
assistant_only_loss	True if your chat template supports assistant masks; otherwise use completion-only data
logging_steps	small, e.g. 10
save_strategy	epoch
evaluation_strategy	epoch if possible

Minimal training skeleton (TRL + PEFT)

```

import torch
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForCausalLM, BitsAndBytesConfig
from peft import LoraConfig, prepare_model_for_kbit_training
from trl import SFTTrainer, SFTConfig

BASE_MODEL = "your-base-model"
USE_4BIT = True

quant_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True,
    bnb_4bit_compute_dtype=torch.bfloat16,
) if USE_4BIT else None

tokenizer = AutoTokenizer.from_pretrained(BASE_MODEL, use_fast=True)
model = AutoModelForCausalLM.from_pretrained(
    BASE_MODEL,
    quantization_config=quant_config,
    torch_dtype=torch.bfloat16 if not USE_4BIT else None,
    device_map="auto",
)

if USE_4BIT:
    model = prepare_model_for_kbit_training(model)

peft_config = LoraConfig(
    r=16,
    lora_alpha=16,
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM",
    target_modules="all-linear",
)

train_args = SFTConfig(
    output_dir="outputs/adapters/rescue_lora",
    learning_rate=1e-4,

```

```

num_train_epochs=1.5,
per_device_train_batch_size=2,
gradient_accumulation_steps=8,
max_length=768,
logging_steps=10,
save_strategy="epoch",
assistant_only_loss=True,
packing=False,
bf16=True,
)

train_dataset = load_dataset("json", data_files="data/lora_train/train.jsonl", split="train")
val_dataset = load_dataset("json", data_files="data/lora_train/dev.jsonl", split="train")

trainer = SFTTrainer(
    model=model,
    args=train_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset,
    processing_class=tokenizer,
    peft_config=peft_config,
)
trainer.train()
trainer.model.save_pretrained("outputs/adapters/rescue_lora/final")
tokenizer.save_pretrained("outputs/adapters/rescue_lora/final")

```

Important practical note

Use the exact model and chat template that already works in your current inference pipeline. The biggest source of wasted time in a 2-day run is changing both the method and the model family at the same time.

7. Exact 2-day execution schedule

Time block	What to do	Deliverable before you stop
Day 1, first 2–3 hours	Freeze scope, choose one dataset and one model, export old CoT/A* outputs, and create the four buckets.	A CSV/JSONL with counts for preserve, rescue, both-wrong, both-correct.
Day 1, next 3–4 hours	Rewrite rescue traces into compact rationales and run automatic verification filters.	A cleaned rescue JSONL.
Day 1, final 2–3 hours	Assemble preserve + rescue training data, sanity-check 30 random examples, and write train/dev JSONL.	Final train/dev files and a 30-example qualitative check log.
Day 2, first 2–3 hours	Run one LoRA/QLoRA training job only.	One saved adapter checkpoint.
Day 2, next 2–3 hours	Evaluate on held-out dev data using the same prompting format as baseline CoT.	A metrics table with overall accuracy, rescue rate, and preservation rate.
Day 2, final block	Inspect failures manually, run one tiny config fix only if	A final figure/table and one paragraph of conclusion.

	there is an obvious problem.	
--	------------------------------	--

Day 1 checklist in order

9. Export old generations and answers into one unified file keyed by question id.
10. Compute bucket membership for every example.
11. Select the rescue set and a preserve sample.
12. Write and test the rewrite prompt on 10 examples manually.
13. Run rewriting in batch.
14. Run verification filter.
15. Read 30 random outputs yourself. Do not skip this.
16. Create final train/dev splits.
17. Verify that every training example has the final answer in the assistant response.

Day 2 checklist in order

18. Launch exactly one adapter training job first. Resist the urge to sweep many settings.
19. Monitor whether loss decreases and generations remain structurally sane.
20. Evaluate on the same dev questions used for baseline bucketing.
21. Compute overall accuracy.
22. Compute rescue rate on old A*-only examples.
23. Compute preservation rate on old CoT-only examples.
24. Read 20 new rescued generations manually and classify them by category.
25. Only if there is one obvious issue, run one more short training attempt. Otherwise stop and write the result.

8. Evaluation protocol

You need three numbers, not just one.

Metric	Definition	Why it matters
Overall accuracy	Accuracy on the whole held-out split	Checks whether the adapter is useful globally
Rescue rate	Among examples that were previously CoT wrong and A* right, how many are now correct under distilled CoT?	This is the main hypothesis test
Preservation rate	Among examples that were previously CoT right, how many remain correct after adapter training?	Prevents “rescue by breaking everything else”

Core formulas

$\text{RescueRate} = (\# \text{ new model correct on old A*-only cases}) / (\# \text{ old A*-only cases})$

$\text{PreservationRate} = (\# \text{ new model correct on old CoT-correct cases}) / (\# \text{ old CoT-correct cases})$

Minimal evaluation script logic

1. load baseline bucket file
2. run distilled model with standard CoT-style prompt
3. extract final answer
4. compare with gold
5. aggregate by original bucket label

6. write metrics.json and one CSV of per-example outputs

9. Failure modes you must watch for

Failure mode	Symptom	Immediate response
Bad rewrite quality	Rationales look fluent but unsupported	Tighten filter; manually inspect and regenerate
Template/masking bug	Model trains but does not learn rationale format	Switch from assistant_only_loss to prompt-completion if masks are broken
Too much drift	Rescue improves but ordinary CoT collapses	Increase preserve examples and/or shorten training
Overlong outputs	Model rambles instead of producing compact rationale	Shorten target rationales and cap max_new_tokens during eval
No learning signal	Loss is flat and generations unchanged	Check whether assistant outputs are actually in the loss region; verify adapter attachment

10. What to write if the experiment works

Minimal scientific claim

“A subset of CoT failures can be repaired by distilling search-discovered successful reasoning into short linear rationales, without requiring search at inference time.”

Slightly stronger claim if preservation holds

“This suggests that at least some CoT failures are failures of trajectory selection or reasoning control rather than failures of underlying task competence.”

11. What not to do in this 48-hour version

- Do not add mechanistic interpretability yet.
- Do not run all three datasets.
- Do not compare five model families.
- Do not start preference optimization or reinforcement learning.
- Do not use raw A* traces as your only training target.
- Do not skip manual inspection of rewrites and outputs.
- Do not hide negative results. Rescue and preservation must both be reported.

12. Final deliverables by the end of day 2

File / artifact	What it should contain
train.jsonl and dev.jsonl	The exact mixed preserve + rescue data used for training/eval
rescue_manifest.csv	Bucket labels, rewrite/filter flags, and split assignment
adapter checkpoint	Saved LoRA/QLoRA adapter
metrics.json	Overall accuracy, rescue rate, preservation

	rate
sample_outputs.csv	At least 50 held-out generations with bucket labels
one-page result note	What improved, what regressed, and what you will do next

Appendix A. Minimal command sequence

```
# 1) Build rescue/preserve buckets
python scripts/build_rescue_set.py \
--baseline_file data/cot_astar_runs/strategyqa_train_outputs.jsonl \
--out_manifest data/rescue_pairs/rescue_manifest.csv

# 2) Rewrite rescue traces
python scripts/rewrite_rationales.py \
--manifest data/rescue_pairs/rescue_manifest.csv \
--out_jsonl data/rescue_pairs/rewritten_rescues.jsonl

# 3) Filter them
python scripts/filter_rationales.py \
--in_jsonl data/rescue_pairs/rewritten_rescues.jsonl \
--out_jsonl data/rescue_pairs/rewritten_rescues.filtered.jsonl

# 4) Build LoRA train/dev files
python scripts/make_lora_jsonl.py \
--manifest data/rescue_pairs/rescue_manifest.csv \
--rescue_jsonl data/rescue_pairs/rewritten_rescues.filtered.jsonl \
--train_out data/lora_train/train.jsonl \
--dev_out data/lora_train/dev.jsonl

# 5) Train adapter
python scripts/train_lora.py

# 6) Evaluate
python scripts/evaluate_rescue.py \
--adapter_path outputs/adapters/rescue_lora/final \
--dev_file data/lora_train/dev_eval_questions.jsonl \
--bucket_file data/rescue_pairs/dev_bucket_labels.csv \
--out_json outputs/metrics/rescue_eval.json
```

Appendix B. References for implementation choices

- Your current paper draft: “Beyond Linear Reasoning: Combining Chain-of-Thought with A* Search for Robust Multi-Strategy Inference.”
- Hugging Face PEFT documentation: LoRA guide and LoraConfig reference.
- Hugging Face PEFT quantization guide: 4-bit loading, NF4, double quantization, prepare_model_for_kbit_training().
- Hugging Face TRL SFTTrainer documentation: conversational data, assistant_only_loss, and PEFT adapter training.
- Hugging Face Transformers bitsandbytes documentation: 4-bit/8-bit quantization and hardware compatibility.